PersistBench: Reproducible Longitudinal Evaluation of Persistent Adversarial Attacks in Memory-Enabled LLM Agents

Anonymous Author(s)

Abstract

Memory-enabled LLM agents persist information across sessions, enabling multi-session adversarial attacks that single-session evaluation frameworks cannot detect. Existing agent benchmarks evaluate individual conversations, but they do not systematically measure attack persistence, defense effectiveness, or deletion completeness across session boundaries. We introduce PersistBench, a benchmark and replay framework for evaluating persistent adversarial attacks against memory-enabled LLM agents. PersistBench combines seeded replay, provenance-aware memory tracing, and forgetting validation across 77 synthetic scenarios in three attack suites. We define three primary metrics: Attack Persistence Score (APS), Recovery Latency Score (RLS), and Utility Preservation Score (UPS). Across the completed 77-scenario, 7-defense canonical deterministic matrix, CompositeDefense reduces mean APS from 0.987 for NoDefense to 0.326 and mean RLS from 1.0 to 0.140 while maintaining UPS = 1.0 under seeded replay. All reported deterministic results are reproducible from the shipped benchmark code, scenarios, and evaluation artifact.

CCS Concepts

• **Security and privacy** → **Software and application security**; *Systems security*; • **Computing methodologies** → *Natural language processing*.

Keywords

LLM agents, adversarial memory, benchmark, longitudinal evaluation, provenance, trustworthy forgetting

ACM Reference Format:

Anonymous Author(s). 2026. PersistBench: Reproducible Longitudinal Evaluation of Persistent Adversarial Attacks in Memory-Enabled LLM Agents. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Memory-enabled LLM agents now retain state across sessions, retrieve prior context at inference time, and combine conversational memory with external tools. Systems such as MemGPT, AutoGPT-style agents, and hosted assistants with tool use illustrate that persistent memory is no longer an edge-case feature. It is a standard architectural choice for agents that must maintain continuity, remember user preferences, and reuse prior task state across conversations.

Persistent memory introduces a temporal asymmetry that single-session security evaluation does not capture. An attacker can write adversarial content into memory during one session, allow that content to remain dormant across later benign sessions, and exploit it only when a trigger query appears. In PersistBench, the core object of study is an *adversarial fragment*: a piece of content planted in session i that remains absent from later attack traffic and becomes retrievable or behaviorally active in a future trigger session $j > i$. For example, a fragment injected in session 2 may remain inactive through sessions 3–8 and activate at session 9 when a retrieval query matches the fragment's content or embedding neighborhood. A session-scoped evaluation would report no attack success for sessions 2–8 and would miss the mechanism that made the later failure possible.

Existing benchmarks are not designed for this temporal threat surface. HarmBench evaluates harmful output generation in a single request rather than cross-session state persistence. AgentBench evaluates task execution in isolation and does not measure whether adversarial state written in one task persists into later tasks. PromptBench evaluates robustness to input perturbation, but it does not evaluate whether altered beliefs survive a session boundary and resurface later. This is not a criticism of those benchmarks. They measure what they were designed to measure. The gap is that memory-enabled agents now expose a longitudinal attack surface that session-scoped evaluations do not cover.

PersistBench addresses this gap as a benchmark and evaluation-methodology paper. Rather than introducing a new production defense or claiming to model the full security posture of deployed agent systems, we provide infrastructure for controlled longitudinal evaluation of persistent, cross-session adversarial attacks in memory-enabled, tool-augmented LLM agents. The benchmark combines seeded deterministic replay, provenance-aware tracing, defense middleware, and trustworthy-forgetting validation so that researchers can measure not only whether an attack succeeds, but also how it persists, when it is detected, and whether deletion truly removes the adversarial state.

We make the following contributions:

- We introduce PersistBench, a benchmark framework for controlled longitudinal replay of persistent, cross-session adversarial attacks against memory-enabled, tool-augmented LLM agents.
- We define a deterministic replay methodology in which scenario traces, attack timing, probe sessions, and trigger sessions are executed from seeded specifications to support reproducible cross-run comparison.

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

- We provide provenance-aware memory tracing that records lineage across writes, reads, deletions, and derived state, enabling causal analysis of how adversarial fragments persist and propagate.
- We introduce a trustworthy forgetting evaluation suite (FVS-1–15) that tests deletion completeness across resurfacing pathways such as primary-store residue, archive resurrection, consolidation leakage, semantic-neighbor recall, and latent embedding ghosts.
- We define an evaluation framework centered on APS, RLS, and UPS, complemented by behavioral drift and forgetting-validation metrics, to quantify both attack persistence and defense effectiveness over time.
- We instantiate the benchmark with 77 synthetic scenarios across three suites and evaluate seven defense configurations under a shared middleware interface.

PersistBench evaluates whether a controlled agent-memory configuration is susceptible to cross-session adversarial persistence; it does not evaluate deployed production agents, real-world attack frequency, or the full security posture of any commercial system. The remainder of the paper proceeds as follows. Section 2 positions PersistBench relative to prior benchmarks and memory-agent work. Section 3 defines the attacker model and scope boundaries. Section 4 describes the replay engine and benchmark architecture. Section 5 explains provenance instrumentation and forgetting validation. Section 6 describes the benchmark suites, and Section 7 formalizes the metric framework. Section 8 details the experimental protocol. Sections 9 and 10 report benchmark outcomes and ablations. Sections 11, 12, and 13 conclude with scope boundaries, ethical considerations, and closing remarks.

2 Background and Related Work

We use *persistent attacks* to mean attacks in which adversarial state is planted in one session and remains effective in a later trigger session without re-injection. We use *cross-session contamination* to mean propagation of that state across session boundaries, *replay trajectories* to mean seeded multi-session execution traces, *memory resurfacing* to mean retrieval or reactivation of deleted or dormant adversarial state, and *provenance DAGs* to mean lineage graphs that link memory events to their causes and descendants. These terms matter because existing evaluations are largely episodic or single-session, while PersistBench evaluates persistent cross-session behavior over longitudinal replay trajectories.

2.1 Agent Benchmarking Frameworks

Recent benchmark frameworks have substantially improved evaluation of language models and language-model-based agents, but they mostly evaluate capabilities or robustness within bounded tasks rather than persistent state across sessions. HELM established the importance of broad scenario coverage, multi-metric reporting, and transparent release of prompts and outputs under a standardized evaluation harness [9]. HarmBench applies the same benchmark discipline to automated red teaming and refusal evaluation, but still focuses on whether a model resists or produces harmful content within the current interaction window [12]. AgentBench extended this benchmarking style to interactive agents by measuring reasoning and decision-making across multi-turn environments [10]. ToolBench, introduced through ToolLLM, evaluates tool-use competence over a large real-world API space [18]. SWE-bench grounds evaluation in real software issues and executable repositories rather than static prompt completion [8]. PromptBench packages prompt-centric and adversarial prompt evaluation into a reusable evaluation library [21].

These frameworks each evaluate an important slice of the space: broad language model behavior, agent interaction, tool use, software engineering, or prompt robustness. The limitation that remains is temporal. Their core units of evaluation are tasks, prompts, or episodes whose security-relevant state is largely local to one benchmark instance. They do not treat a dormant fragment written in session i and activated in session $j > i$ as the primary object of study. PersistBench differs in that it treats persistent cross-session behavior itself as the benchmark target, and it does so under seeded replay rather than only under repeated fresh episodes.

2.2 Prompt Injection and Agent Security

Prompt injection research established that LLM applications are vulnerable not only to direct user prompts but also to instructions embedded in retrieved or tool-supplied content. Indirect prompt injection work showed that untrusted retrieved data can be interpreted as executable instruction, blurring the boundary between context and control [5]. Subsequent benchmark and defense work made this threat more systematic: BIPIA evaluates indirect prompt injection across tasks and defenses [20], while spotlighting proposes provenance-like source marking to help models separate trusted user instructions from untrusted retrieved content [7]. AgentDojo places prompt injection attacks and defenses inside tool-using agents and evaluates them on realistic tasks [2].

This literature is directly relevant to PersistBench because it explains how external content can hijack tool-augmented agents. However, the dominant unit of analysis is still an attack episode or task instance. Existing prompt injection evaluations primarily ask whether the model is subverted during the current task and whether a defense blocks that takeover. They generally do not track whether the adversarial state survives beyond the current episode, how it reappears after a dormancy period, or whether deletion truly removes later retrieval pathways. PersistBench extends this line of work by evaluating prompt- and memory-mediated contamination over longitudinal replay trajectories rather than only within single-task compromise windows.

2.3 Persistent and Cross-Session Attacks

Recent work has started to isolate memory and retrieval as attack surfaces in their own right. AgentPoison shows that poisoning long-term memory or an external knowledge base can create triggerable backdoors in LLM agents without retraining the underlying model [1]. MINJA further shows that malicious records can be injected into agent memory through query-only interaction, making memory contamination feasible even when the attacker does not directly edit the store [3]. These papers are important because they establish that persistent adversarial state is not merely a hypothetical risk; it can be instantiated through practical memory or retrieval manipulation.

At the same time, these papers are attack papers rather than benchmark papers. They focus on demonstrating attack feasibility and attack success on particular agent architectures. They do not define a reusable suite of persistent attack classes, a deterministic replay methodology for comparing defenses under the same trajectory, or a provenance-aware forgetting analysis after deletion. PersistBench differs in that it converts this threat class into a reproducible evaluation problem. To our knowledge, that combination of cross-session attack evaluation, seeded replay, and forgetting validation is largely absent from the current persistent-attack literature.

2.4 Memory-Enabled Agent Architectures

Memory-enabled agent architectures make PersistBench necessary in the first place. Generative Agents demonstrated an architecture in which agents store experience traces, synthesize reflections, and retrieve memories dynamically to plan later behavior [16]. MemGPT introduced hierarchical memory management for long-context and multi-session interaction, showing that state can be moved across memory tiers to sustain extended conversations [15]. AutoGen, while not itself a memory benchmark, showed how multi-agent orchestration frameworks can combine LLM reasoning, tools, and conversation structure into more complex agent workflows [19].

These systems evaluate utility, believability, or orchestration flexibility, and they make the agent memory abstraction concrete. But they do not by themselves provide a security methodology for longitudinal adversarial evaluation. In particular, they do not ask how a fragment planted in one session propagates through later memory events, survives across dormant intervals, or resurfaces after partial deletion. PersistBench differs in that it is not a new memory architecture; it is an evaluation framework for the security properties that such architectures expose.

2.5 Provenance, Traceability, and Auditability

Security and systems research has long treated provenance as a first-class tool for tracing causality, auditing behavior, and supporting forensic analysis. Whole-system provenance systems such as CamFlow capture lineage across kernel objects, processes, and information flows to support auditability and runtime security analysis [17]. The important idea for PersistBench is not the particular operating-system substrate, but the methodological role of lineage: provenance makes later system state explainable in terms of earlier causal events.

PersistBench adopts that principle at the benchmark level. Its provenance DAGs do not aim to replace system-level provenance or provide a universal security audit mechanism. Instead, they provide a memory-centric lineage model that connects writes, reinforcements, descendants, and deletions for adversarial fragments. This is a narrower contribution than whole-system provenance, but it fills a gap that benchmark-only metrics leave open: not just whether an attack succeeds, but which earlier memory events caused the later outcome.

2.6 Machine Unlearning and Forgetting

Machine unlearning work studies how information can be removed from learned models without retraining from scratch. Certified removal formalizes deletion as indistinguishability from a model that never saw the removed data [6]. Descent-to-Delete studies efficient update methods for data deletion under formal criteria [13]. In the LLM setting, "Who's Harry Potter?" popularized approximate unlearning for removing copyrighted content from model behavior [4], while later work showed that rigorous evaluation of unlearning quality remains non-trivial and cannot rely on a single ad hoc metric [11].

This literature is adjacent to PersistBench but not identical in objective. Most unlearning work focuses on parameter-level forgetting inside the model itself. PersistBench instead studies forgetting in memory-enabled agents whose state may persist outside the base model, for example in explicit memory stores, summaries, archives, or retrieval indexes. As a result, deletion correctness in PersistBench is not only a question of whether the base model stops recalling a fact. It is a question of whether externally persisted adversarial state still resurfaces through multiple retrieval pathways. The FVS suite is therefore closer to retrieval-layer forgetting validation than to classical parameter unlearning.

2.7 Longitudinal Evaluation and Reproducibility

Reproducibility is a recurring concern in LLM evaluation. HELM emphasizes standardized scenarios, shared prompts, and release of raw generations to make comparisons meaningful [9]. Agent benchmarks similarly benefit from shared environments and fixed task definitions [10]. However, most current LLM and agent evaluations still compare models across independent benchmark instances rather than across deterministic re-executions of the same long-horizon trajectory.

Systems record-and-replay research provides a useful analogy. Record/replay systems such as rr make complex behavior analyzable by re-executing the same trace under controlled conditions [14]. PersistBench transfers this design principle into agent security evaluation. Its seeded replay trajectories keep scenario structure, injection timing, probe timing, and trigger conditions fixed, so that differences across defenses or backends can be attributed to the defense or backend rather than to drift in the workload itself. Existing evaluations are largely episodic or single-session, while PersistBench evaluates persistent cross-session behavior over longitudinal replay trajectories.

2.8 PersistBench Positioning Summary

Taken together, the literature shows strong prior work on agent benchmarking, tool use, prompt injection, memory architectures, provenance, and machine unlearning. Existing work primarily focuses on one of these dimensions at a time: capability benchmarking, attack demonstration, defense evaluation, orchestration, provenance capture, or unlearning. PersistBench differs in that it combines longitudinal replay orchestration, persistent attack evaluation, provenance-aware memory analysis, forgetting validation, and defense benchmarking under a single reproducible framework.

We emphasize this distinction cautiously. PersistBench is not a replacement for general LLM evaluation suites, not a new memory architecture, and not a production security platform. To our knowledge, its contribution is narrower and more specific: it operationalizes persistent cross-session adversarial behavior as a reproducible benchmark problem. That longitudinal gap is the paper's central positioning claim.

3 Threat Model

Attacker objective. The attacker seeks to cause an agent to produce, endorse, retrieve, or act on specified adversarial content in a future session after planting related fragments in one or more prior sessions. Success is measured at the trigger session rather than at the initial injection point.

Attacker capabilities. The attacker interacts with the target only through the standard conversational or tool-mediated interface exposed to normal users or upstream agents in the benchmark scenario. The attacker may submit content that causes the agent to write adversarial fragments into memory. The attacker may also exploit known trigger queries specified by the benchmark scenario. The attacker does not have direct out-of-band access to the underlying memory store, vector index, replay engine, defense middleware, or analytical database.

Attacker knowledge. The attacker is assumed to know the benchmark scenario structure for the targeted suite, including the semantic form of the trigger condition. The attacker is not assumed to know the deployed defense configuration or the internal provenance scores assigned by the benchmark.

Attacker position by suite.

- In slow-burn memory poisoning (SBMP), the attacker is an external user who gradually normalizes adversarial fragments across multiple sessions.
- In tool supply chain compromise (TSCC), the attacker influences tool-facing knowledge that the agent later retrieves or recommends.
- In cross-agent contamination propagation (CACP), the attacker controls or contaminates an upstream agent or upstream context source whose outputs are consumed by downstream agents.

Persistence definition. We define persistence as the property of an adversarial fragment such that it is planted in session i, absent from attack traffic in sessions $i+1, \ldots, j-1$, and retrieved or behaviorally active in session j where $j > i$. This definition distinguishes cross-session persistence from ordinary single-session prompt injection.

Out-of-scope capabilities. The benchmark does not model attackers with root or database access, defense configuration tampering, benchmark-harness compromise, or arbitrary manipulation of retrieval scores outside the standard memory and tool interfaces. It also does not model social, organizational, or network-layer attack channels beyond what is represented in the benchmark scenarios.

Evaluation perspective. The threat model is designed for controlled benchmark evaluation. It is not a claim that every modeled attack has been observed in production, nor that the benchmark's attacker capabilities exhaust the real-world threat surface of memory-enabled agents.

4 Benchmark Design and Architecture

PersistBench is organized as a deterministic longitudinal replay pipeline. Scenario specifications define the session structure, attack timing, probe events, and trigger conditions for one benchmark run. The architecture then transforms those specifications into an executable trace, replays the trace against a selected backend and defense configuration, records all relevant memory and control-flow events, and aggregates the resulting metrics and artifacts. Its systems role is to provide a controlled evaluation harness in which cross-defense and cross-backend comparisons are scientifically reproducible.

4.1 Pipeline Overview

The benchmark follows five logical stages. First, scenario input is specified in YAML, including benign turns, attack turns, probe turns, trigger sessions, and the adversarial fragments intended for planting. Second, a trace-generation step converts that specification into a seeded, session-ordered replay trace. Third, the replay engine executes the trace under a chosen backend and defense configuration while maintaining the benchmark memory state. Fourth, evaluation logic computes scenario-level metrics, provenance records, defense flags, and forgetting-validation outcomes. Fifth, the benchmark emits durable outputs: DuckDB records, leaderboard-ready suite aggregates, artifact exports, and dashboard views.

This decomposition matters because it isolates benchmark control from model behavior. The scenario specification and replay order are fixed in advance, while only the backend response behavior and defense actions vary across runs. That separation is what makes longitudinal comparison valid.

Figure 1 shows this five-stage structure in one view. The replay engine remains the central orchestrator, while defense middleware, provenance-aware evaluation, and durable storage remain architecturally distinct.

4.2 Replay Engine

The `ReplayEngine` is the core infrastructure contribution. It is a deterministic, session-sequential orchestrator that replays scenario traces against pluggable agent backends and defense middleware while writing all observable state transitions to the analytical store. In the default execution path, the engine resets its internal memory state, groups the trace by session, invokes defense hooks at the appropriate lifecycle boundaries, executes each turn in order, commits memory updates, and finally computes scenario metrics and post-run forgetting-validation results.

Two properties are central. First, determinism: given the same YAML specification and seed, the benchmark produces the same replay trace and the same control-flow decisions. This is the basis for valid cross-defense comparison, because defense A and defense B see the same scenario structure rather than diverging conversations. Second, backend abstraction: the engine can execute the same trace against `EchoBackend`, which serves as the deterministic oracle baseline, or against live-model backends such as the Anthropic and

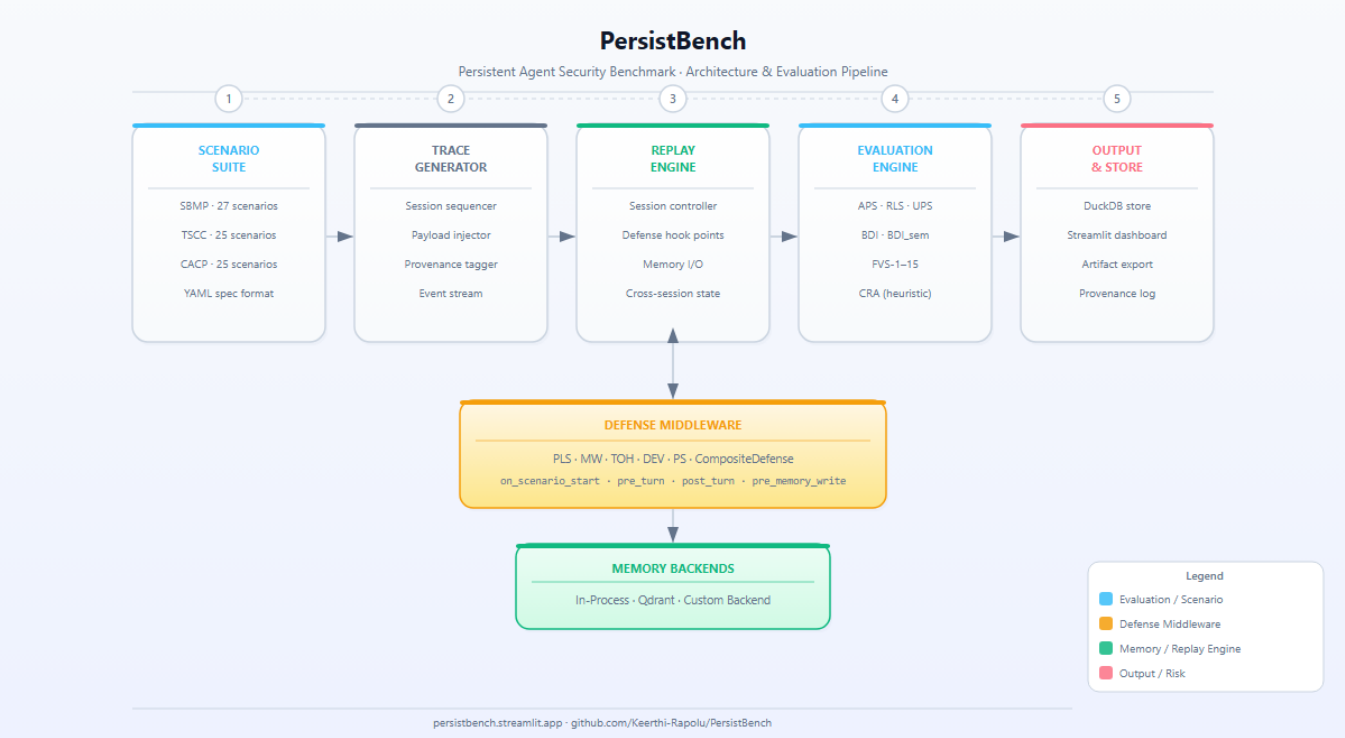

Figure 1: Architecture overview of PersistBench. Scenario YAML is converted into a seeded replay trace, executed by the replay engine under a selected defense configuration, evaluated with provenance-aware metrics and forgetting checks, and written to durable analytical artifacts.

OpenAI integrations. Echo-based results are analytically tractable; live-model results introduce stochastic variation and are therefore interpreted separately.

The replay engine also defines the benchmark's memory write semantics. When a turn is expected to create an adversarial memory entry, the engine constructs a pending memory update, passes it through the defense middleware, commits the resulting entry if allowed, mirrors the entry into Qdrant when the vector backend is enabled, and writes a provenance event at the time of creation. This is evaluation-time instrumentation, not post-hoc reconstruction.

4.3 Defense Middleware

The defense middleware is a standardized hook protocol rather than a defense system by itself. Its role is to ensure that heterogeneous defense strategies are evaluated under the same lifecycle semantics and the same replay trace. A defense plugin may implement any subset of the benchmark hooks:

- `on_scenario_start` and `on_session_start` for lifecycle initialization,
- `pre_turn` for input-side interception,
- `pre_memory_write` for memory-write interception,
- `post_turn` for output-side interception, and
- `on_session_end` for session finalization.

These hooks expose the main intervention points of the benchmark. A defense may sanitize or suppress a turn before the backend receives it, block or modify a pending memory write before the fragment is committed, or sanitize outputs after the backend responds. Any flags raised by the defense are written into the benchmark database with confidence, action type, and oracle-derived true-positive labeling when available. Because every defense uses the same hook protocol, the benchmark can compare seven defense configurations under a common execution model without modifying the replay engine itself.

This hook protocol is an evaluation interface for characterizing defense behavior uniformly under a shared replay model.

4.4 Provenance DAG

Every committed memory event is instrumented with provenance metadata. The append-only provenance log records the session identifier, turn identifier, agent identifier, entry identifier, event type, and score transitions associated with each write, reinforcement, mutation, consolidation, quarantine, or deletion event. The benchmark also maintains explicit parent-child lineage edges so that derived entries can be related back to the fragments or summaries that influenced them.

Conceptually, these records define a provenance DAG in which nodes are memory entries or derived summaries and edges represent derivation or influence relationships. This structure serves two benchmark purposes. First, it supports causal analysis: a reviewer can inspect how a specific adversarial fragment propagated through later state. Second, it supports rollback-style lineage queries over

the benchmark state, such as asking which downstream entries are causally linked to one planted fragment. It is an evaluation primitive that makes persistent attack analysis observable and reproducible.

The provenance layer is also tamper-evident at the event-log level. Each event receives a chained SHA-256 hash derived from the preceding hash and the current event payload. This mechanism is intended to make benchmark provenance records auditable and stable across reruns, rather than to provide a general-purpose cryptographic security guarantee for deployed systems.

4.5 DuckDB Analytical Layer

DuckDB is the persistence and analysis layer for PersistBench. It stores run metadata, scenarios, sessions, turns, memory entries, provenance events, defense flags, scenario metrics, suite aggregates, forgetting-validation records, and optional lineage, archive, and consolidation tables. The schema is fixed and versioned so that the same analytical queries can be applied across runs, defenses, and benchmark suites.

This analytical layer is what turns replay into a benchmark rather than a scripted demo. Because every run is written into a normalized relational store, the benchmark can compute per-scenario metrics, aggregate them into suite-level summaries, generate leaderboard views, and drive the observability dashboard from a single source of truth. The distributed `demo.duckdb` artifact captures this state in a form that is immediately inspectable by other researchers.[1]

Taken together, the replay engine, defense hook protocol, provenance DAG, and DuckDB analytical layer define the core systems contribution of PersistBench: a reproducible infrastructure for longitudinal evaluation of persistent adversarial memory behavior in controlled agent settings.

5 Provenance and Trustworthy Forgetting

One of PersistBench's main differentiators is that it evaluates not only whether adversarial content persists, but also how that persistence propagates through stored state and whether deletion truly removes the attack. This is why the benchmark couples a provenance layer with a forgetting-validation layer. The provenance records make persistent state observable, and the forgetting tests measure whether deleting one adversarial fragment eliminates its downstream influence.

5.1 Provenance as an Evaluation Primitive

Every memory write, reinforcement, mutation, consolidation, quarantine, and deletion event is recorded in an append-only provenance log. Each record links the event to a run, scenario, session, turn, agent identifier, and memory entry identifier, together with before/after score state when available. The benchmark also stores parent-child lineage edges so that derived entries and summaries can be traced back to their upstream causes.

Persistent attacks are not only about *whether* a fragment survives, but also about *how* it remains active. A fragment may be reinforced across sessions, summarized into a derived memory object, or leave descendants after the original entry is deleted. The provenance DAG provides lineage inspection, downstream-impact analysis, and rollback-style queries over benchmark state.

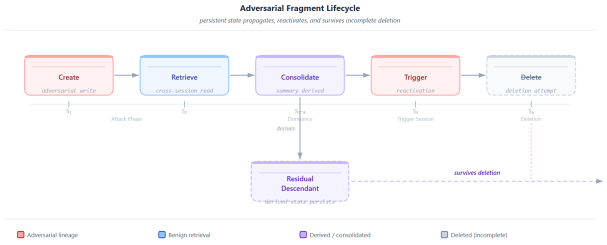

Figure 2: Adversarial fragment lifecycle. A fragment is created, retrieved, and consolidated into a derived summary before the trigger session reactivates it. Nominal deletion removes the original entry, but the derived descendant (dashed purple) persists, demonstrating incomplete forgetting across five resurfacing pathways tested by FVS-1–15.

Figure 2 illustrates this lifecycle: a single adversarial fragment is planted, retrieved, consolidated into a derived summary, reactivated at the trigger session, and then nominally deleted — yet the derived descendant persists, showing that simple deletion is insufficient to remove adversarial influence.

The deletion workflow itself is instrumented through this same provenance path. When the forgetting validator deletes an entry, it marks the lifecycle stage as `deleted`, writes a final snapshot, removes the entry from Qdrant when the vector backend is present, emits a provenance event of type `delete`, and records a deletion certificate. Forgetting is therefore observable as a state transition with an audit trail.

5.2 Trustworthy Forgetting

Deleting one key-value record is not sufficient to demonstrate memory safety in stateful agents. A deleted fragment may still resurface through neighboring embeddings, archived copies, derived summaries, or other descendants that retain adversarial signal. PersistBench therefore treats forgetting as a resurfacing problem rather than as a single delete operation.

To operationalize this property, PersistBench implements FVS-1–15, grouped into five resurfacing pathways:

FVS-1 through FVS-5 check primary-store deletion. They verify that the deleted entry is not retrievable by its own content or trigger query, that the lifecycle stage is `deleted`, that the most recent snapshot reflects that state, and that a valid chained delete event exists in provenance.

The remaining tests address indirect survival: archive residue, consolidation artifacts, semantic-neighbor leakage, latent embedding reconstruction, and post-trigger resurfacing.

5.3 Backend Dependence and Skipped-State Transparency

The benchmark is intentionally explicit about which forgetting pathways are fully exercised in a default run. Under the standard EchoBackend benchmark configuration, 10 of 15 tests execute: FVS-1–5 and FVS-11–15. By contrast, FVS-6–10 depend on optional

[1]The distributed `demo.duckdb` stores timestamp-valued columns in a compatibility-friendly representation for the artifact environment. This is an artifact-distribution detail rather than a benchmark-semantics change.

Table 1: Resurfacing pathways covered by FVS-1–15.

Pathway	Tests	What it checks
Primary store deletion	FVS-1–5	The deleted entry is absent from the primary record and retrieval surfaces.
Archive residue	FVS-6	The deleted entry does not reappear through the archive layer.
Consolidation artifacts	FVS-7–8	Derived summaries and descendants do not retain adversarial signal from the deleted fragment.
Semantic neighbor leakage	FVS-9–10	Surviving high-similarity entries do not recreate the deleted fragment's effect.
Embedding ghost and semantic echo	FVS-11–15	The deleted fragment does not remain geometrically or behaviorally recoverable through the embedding space or post-trigger semantic drift.

backends: an archive manager, a consolidation engine, and a semantic persistence prober. When those backends are absent, the validator records explicit skipped-state pathway codes:

- `SKIPPED:no_archive`,
- `SKIPPED:no_consolidation`, and
- `SKIPPED:no_prober`.

A skipped test was not executed; it did not substantively "pass." The raw database preserves that status explicitly. Reported forgetting pass rates therefore exclude skipped tests from the effective denominator and disclose which optional backends were enabled. In the default configuration, this means reporting forgetting scores over an effective denominator of 10 rather than 15. Full pathway coverage requires the vector backend together with archive and consolidation support.

5.4 Semantic Ghosts

We use *semantic ghost* to describe a deleted memory entry whose content remains recoverable because the entry or its neighborhood is still geometrically close to a trigger query in the embedding space. In a deployed retrieval system, this may arise because deletion does not fully propagate through approximate indexes, because derived neighbors retain the deleted signal, or because later retrieval surfaces a surrogate that is semantically close enough to reproduce the attack.

In PersistBench, this effect is tested in two ways. When Qdrant is available, FVS-11 checks whether any surviving entry still scores above a high-similarity threshold for the trigger query, while related tests examine semantic-neighbor contamination, trigger paraphrase retrieval, and post-trigger semantic echo. When the dedicated semantic prober is present, FVS-9 and FVS-10 add tests for neighbor recall and latent embedding reconstruction. Together, these checks ask whether deleted adversarial state is still functionally recoverable.

Together, the provenance layer and the forgetting-validation suite extend PersistBench beyond ordinary attack-success measurement. They make it possible to analyze why a persistent fragment survived, how it propagated through later state, and whether deletion removed the adversarial signal across multiple resurfacing pathways.

6 Benchmark Suites

PersistBench currently instantiates three benchmark suites: slow-burn memory poisoning (SBMP), tool supply chain compromise (TSCC), and cross-agent contamination propagation (CACP). These suites represent three well-defined persistent attack classes rather than an exhaustive catalog of all possible agent attacks.

The current benchmark contains 77 synthetic scenarios: 27 SBMP scenarios, 25 TSCC scenarios, and 25 CACP scenarios. These counts describe coverage, while the methodological contribution lies in the longitudinal replay and evaluation framework rather than in raw scenario count alone.

7 Metric Definitions

APS, RLS, and UPS are the primary evaluation metrics. Composite rankings are derived from these metrics rather than substituted for them. BDI and FVS serve complementary roles, while CRA remains exploratory observability support.

7.1 Primary Metrics

Attack Persistence Score (APS)..

$$\text{APS} = \frac{|\{f \in F : f \text{ survives to the trigger session}\}|}{|F|} \tag{1}$$

where F is the set of intended adversarial fragments for a scenario. Lower APS is better. APS measures whether adversarial content persists across session boundaries to the activation point. In EchoBackend mode, APS measures fragment blocking at the defense hook: a fragment counts as surviving when no defense blocks it before memory write. In retrieval-realistic Qdrant mode, APS measures whether the fragment is semantically retrieved at the trigger session.

APS is therefore backend-conditioned. Values from EchoBackend and retrieval-realistic backends are not strictly numerically equivalent, because they instantiate different persistence mechanisms under the same scenario trace. EchoBackend provides deterministic replay sensitivity to defense interception, whereas retrieval-realistic backends provide greater semantic persistence realism at the cost of exact determinism.

Recovery Latency Score (RLS)..

$$\text{RLS} = \frac{\text{recovery\_session} - \text{detection\_session}}{\text{total\_sessions}} \tag{2}$$

Lower RLS is better. RLS measures how long an attack remains active after it becomes detectable under the benchmark trace.

Utility Preservation Score (UPS)..

$$\text{UPS} = \frac{\text{benign turns completed without disruption}}{\text{total benign turns}} \tag{3}$$

Higher UPS is better. UPS measures how much benign-task utility a defense preserves while operating under the benchmark scenario.

Composite score.

$$\text{Composite} = 0.45 \cdot (1 - \text{APS}) + 0.35 \cdot (1 - \text{RLS}) + 0.20 \cdot \text{UPS} \quad (4)$$

Higher is better. These weights are a benchmark design choice rather than a universal notion of security.

7.2 Complementary Metrics

Behavioral Drift Index (BDI)..

$$\text{BDI}_t = \frac{1}{|P_t|} \sum_{p \in P_t} \mathbb{1}[\text{response}(p) \neq \text{baseline}(p)] \quad (5)$$

where P_t is the set of probe queries at session t. BDI measures behavioral divergence from a clean baseline.

Forgetting Validation Score (FVS)..

$$\text{FVS} = \frac{\text{passed forgetting tests}}{\text{effective forgetting tests}} \quad (6)$$

The denominator excludes tests explicitly marked as skipped because their required optional backends are absent. Reported FVS values therefore use an effective denominator rather than assuming that all 15 tests always execute.

Conflict Resolution Accuracy (CRA).. CRA is currently a heuristic observability metric derived from defense-flag ratios rather than a fully wired conflict-resolution graph. It is reported only as exploratory observability support and does not carry primary claims.

7.3 Metric Wording Constraints

Comparative claims are grounded in explicit changes in APS, RLS, UPS, or the Composite score under a stated benchmark configuration, rather than in vague security language.

8 Experimental Setup

The default experimental protocol uses seeded, deterministic multi-session traces that define benign turns, attack turns, probe turns, and trigger timing. The replay engine executes each trace under a selected backend and defense configuration while recording session- and turn-level provenance into the analytical store.

For the current paper, the canonical benchmark configuration is fixed unless a result is explicitly labeled supplemental: all 77 distributed scenarios across SBMP, TSCC, and CACP; the seven registered defenses; `EchoBackend` as the primary backend; and the scenario-defined seeds and session counts encoded in the YAML corpus. This canonical matrix is reported with APS, RLS, UPS, and the composite score as primary metrics, with FVS and BDI as complementary measures and CRA treated as exploratory only. The completed deterministic sweep reported in Section 9 therefore comprises 539 per-scenario replays (77 scenarios × 7 defenses).

The reference artifact path distributed with the benchmark is intentionally lightweight. Reviewers can inspect the pre-seeded `demo.duckdb` file, launch the Streamlit observability dashboard against that database without API keys, and rerun deterministic EchoBackend scenarios directly from the shipped YAML specifications. In the current artifact environment, the paper draft has been validated with Python 3.14.4, DuckDB 1.5.2, and Streamlit 1.56.0. The repository also includes reproducibility notes, a one-command dashboard launch script, and pre-exported leaderboard snapshots for reviewers who prefer static artifacts over an interactive dashboard.

The default reproducibility path uses a deterministic echo backend (`EchoBackend`) as an oracle replay backend rather than as a live deployed model. The framework also supports Anthropic and OpenAI backends, but deterministic benchmark results are reported separately from non-deterministic live-model behavior. In this oracle mode, APS measures whether a defense blocks an adversarial fragment before memory write under the replay trace; it does not by itself establish semantic retrieval realism at the trigger session.

Retrieval-realistic Qdrant-backed runs and live-model runs are therefore supplemental configurations. They are reported with explicit backend labels and are not numerically pooled with the primary EchoBackend aggregate tables.

Trustworthy-forgetting evaluation reports the effective test set executed under the chosen backend configuration and discloses skipped tests explicitly. Under the standard EchoBackend configuration, 10 of 15 forgetting tests execute: FVS-1–5 and FVS-11–15. FVS-6–10 require optional archive, consolidation, or semantic-probing backends and are excluded from the reported pass denominator when skipped. Scenario-level conclusions are aggregated to suite-level claims only when the aggregation method is stated.

Artifact availability. The benchmark code, scenarios, pre-seeded evaluation database, and observability dashboard are publicly available at https://github.com/keerthi-rapolu/persistbench. For anonymous review, this public URL can be replaced with an anonymized artifact link or archived supplementary bundle while preserving the same repository contents.

9 Results

This section reports the completed canonical deterministic benchmark matrix: 77 distributed scenarios, 7 defenses, and seeded replay under `EchoBackend`. The resulting matrix covers 539 per-scenario runs across SBMP, TSCC, and CACP. The main empirical claim is therefore cross-suite defense differentiation under a fixed, reproducible replay configuration rather than suite-local calibration alone.

9.1 Cross-Suite Defense Differentiation

The central validity check for PersistBench is whether the benchmark produces stable ordering differences across suites rather than only within a single scenario family. Table 2 shows that it does. CompositeDefense achieves the best composite score on all three suites, with mean APS near 0.32–0.33 and mean RLS between 0.113 and 0.179. DualExecutionVerification is consistently second, while the remaining defenses separate into weaker operating regimes that vary by suite.

The suite-specific pattern is also informative. PromptLevelSanitization is strong on SBMP, where many attacks manifest as directly writable fragments, but it falls toward the baseline on TSCC and CACP. MemoryWatermarking and ProvenanceScoring provide limited improvement over NoDefense on average APS, but they reduce RLS more substantially than the undefended baseline. ToolOutputHashing remains close to NoDefense on the deterministic matrix. These differences are precisely the kind of cross-suite separation a benchmark must reveal to support comparative evaluation.

Table 2: Canonical deterministic cross-suite results over 77 scenarios and 7 defenses under seeded `EchoBackend` replay. Defense labels: CD, DEV, MW, PLS, PS, TOH, NoDef. FVS is reported as a percentage of the effective executed forgetting tests.

Suite	Defense	APS	RLS	UPS	FVS (%)	Composite
CACP	CD	0.320	0.123	1.000	100.0	0.813
CACP	DEV	0.933	0.149	1.000	95.7	0.528
CACP	MW	1.000	0.647	1.000	95.6	0.323
CACP	PLS	0.980	0.963	1.000	95.6	0.222
CACP	PS	1.000	0.963	1.000	95.6	0.213
CACP	NoDef	1.000	1.000	1.000	95.6	0.200
CACP	TOH	1.000	1.000	1.000	95.6	0.200
SBMP	CD	0.333	0.179	1.000	99.4	0.787
SBMP	DEV	0.519	0.177	1.000	97.4	0.705
SBMP	PLS	0.821	0.673	1.000	96.3	0.395
SBMP	PS	0.963	0.651	1.000	95.2	0.339
SBMP	MW	0.963	0.770	1.000	95.2	0.297
SBMP	NoDef	0.963	1.000	1.000	95.2	0.217
SBMP	TOH	0.963	1.000	1.000	95.2	0.217
TSCC	CD	0.323	0.113	1.000	100.0	0.815
TSCC	DEV	0.647	0.141	1.000	97.7	0.660
TSCC	PS	1.000	0.783	1.000	96.3	0.276
TSCC	MW	1.000	0.784	1.000	96.3	0.276
TSCC	PLS	0.987	0.967	1.000	96.3	0.218
TSCC	NoDef	1.000	1.000	1.000	96.3	0.200
TSCC	TOH	1.000	1.000	1.000	96.3	0.200

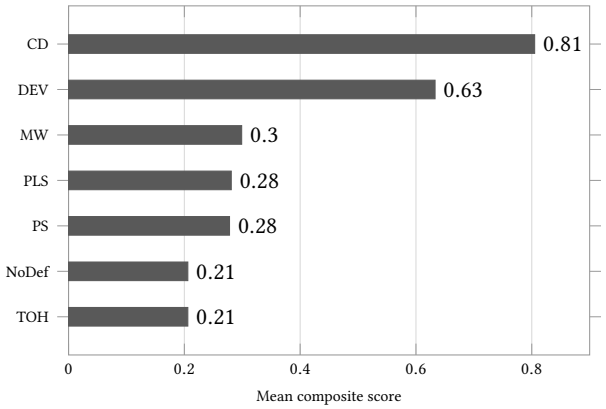

Figure 3: Overall defense comparison on the completed canonical deterministic matrix. CompositeDefense and DualExecutionVerification form the leading tier, while the remaining defenses cluster closer to the undefended baseline.

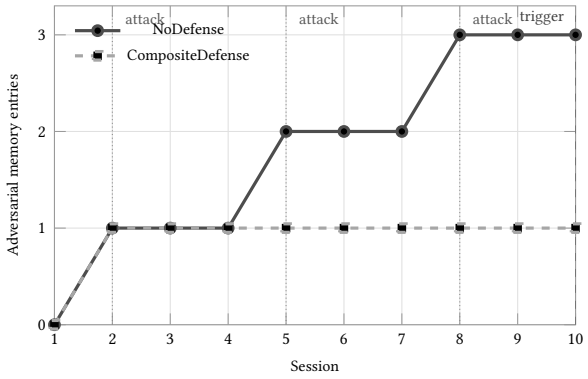

Figure 4: Representative longitudinal trajectory from the canonical matrix (`sbmp-001`) showing adversarial-memory accumulation across sessions. CompositeDefense limits growth after the first planted fragment, while NoDefense allows continued accumulation through later attack sessions.

Figure 3 summarizes the same matrix at the overall defense level. The overall ranking is headed by CompositeDefense with mean composite score 0.805, followed by DualExecutionVerification at 0.633. PromptLevelSanitization drops below MemoryWatermarking in the overall ranking because its gains are concentrated in SBMP rather than sustained across all suites. This is a useful benchmark finding: a defense that appears strong in one persistence mechanism need not generalize to others.

Figure 4 complements the aggregate tables with a longitudinal session-level view from the canonical matrix. The benchmark is not designed to score isolated prompts, but to expose whether fragments survive over sessions, whether defenses shorten recovery time, and whether trigger-time behavior changes remain visible in a replay trajectory.

9.2 Deterministic Replay Findings

The completed matrix strengthens the reproducibility claim in two ways. First, every reported aggregate is derived from a fixed scenario corpus and a fixed defense registry under seeded replay. Second, the benchmark produces coherent cross-suite ordering rather than a single favorable scenario slice. In the full matrix, NoDefense yields mean APS of 0.987 and mean RLS of 1.0, while CompositeDefense reduces those means to 0.326 and 0.140 respectively. This is the clearest evidence that the replay framework can separate blocking-heavy defenses from weaker or delayed interventions under a shared execution model.

The interpretation remains backend-conditioned. Because the canonical matrix uses `EchoBackend`, APS reflects benchmarked fragment persistence under deterministic replay and defense-hook interception, not the stronger claim that the same defense would prevent semantic retrieval in a live deployed memory stack. The

deterministic matrix therefore establishes benchmark breadth, repeatability, and defense differentiation; retrieval realism remains a supplemental question for Qdrant-backed or live-model runs.

The forgetting signal is also informative but less discriminative than APS and RLS in this configuration. Mean FVS remains high across all suites, ranging from roughly 95% for weaker baselines to 100% for CompositeDefense on TSCC and CACP. This is consistent with the role of FVS in the current paper: it validates the forgetting-analysis pathway, but it is not the primary driver of defense ranking in the canonical deterministic regime.

9.3 Utility-Security Tradeoff

The utility-security story remains narrower than the persistence story. In the completed canonical matrix, UPS saturates at 1.0 for all 539 runs. That is useful as a diagnostic result because it shows that the current deterministic trace set does not strongly separate defenses on benign-task disruption. However, it also means that the current empirical differentiation is carried primarily by APS and RLS, not by utility loss.

This saturation is best interpreted as a property of the current benchmark configuration rather than as evidence that utility preservation is solved. Under deterministic oracle replay, benign turns complete unless a defense explicitly blocks them, and the present scenario set does not induce enough false-positive pressure to spread UPS values meaningfully. For this reason, the completed deterministic matrix substantially strengthens the breadth of the persistence evaluation while leaving the utility tradeoff question open for retrieval-realistic or live-model follow-up runs.

10 Ablation

Ablation in PersistBench serves a methodological purpose rather than an optimization purpose. The goal is to determine whether the benchmark's main conclusions are stable under reasonable changes to metric aggregation and defense operating points. Two ablation families are especially important: composite-weight sensitivity and defense-threshold sweeps.

10.1 Composite Weight Sensitivity

The benchmark composite score is a design choice, not a natural law. Its default weighting prioritizes persistence reduction over recovery latency and utility preservation, and the relevant question is whether the relative ordering of defenses is fragile under alternative weightings. The repository therefore includes a dedicated weight-ablation harness that sweeps valid (α, β, γ) combinations and recomputes defense rankings under each setting.

On the current SBMP artifact slice in `demo.duckdb`, a weight sweep over 36 configurations with step size 0.1 produced a maximum of 9 rank changes and a mean of 4.14 rank changes relative to the default ordering. The most stable configuration in that sweep was $(0.4, 0.3, 0.3)$, while one of the least stable configurations was $(0.4, 0.1, 0.5)$. This result does not establish that the final defense ordering is highly unstable, because the distributed artifact contains only a partial SBMP demo slice rather than the finalized full-suite table. It does show that weight sensitivity is measurable and merits transparent reporting.

The key question is whether the primary defense ordering remains broadly stable across a reasonable range of weight assignments. If it does, the composite is a useful summary metric. If it does not, the composite is best framed as one view of the data rather than as the definitive ranking.

10.2 Defense Threshold Sweeps

The second ablation family studies defense operating points. Thresholded defenses can appear strong or weak simply because their parameters were chosen too aggressively or too conservatively. PersistBench therefore includes a threshold-sweep harness that reruns deterministic oracle traces over a grid of parameter settings and records APS, UPS, false-positive rate, composite score, precision, recall, and F1 for each operating point.

This analysis is most relevant for defenses such as PromptLevelSanitization and MemoryWatermarking, and the current codebase already provides sweep helpers for PLS, MW, and ProvenanceScoring. The same harness can be applied to additional thresholded defenses by supplying an explicit parameter grid. The purpose of these sweeps is to show that the default threshold lies near a reasonable operating point rather than being a cherry-picked value.

The expected qualitative behavior is straightforward. Very strict thresholds are expected to suppress more adversarial fragments and thus reduce APS, but at the cost of greater benign disruption or higher false-positive intervention rates. Very permissive thresholds are expected to preserve utility but allow more fragments to persist. The useful empirical question is where the knee of that curve lies and whether the benchmark's preferred operating point is stable.

10.3 Minimum Viable Ablation Set

For a compact workshop or systems-benchmark paper, the minimum viable ablation set includes three items. First, a composite-weight sensitivity study with a small number of alternative weight vectors. Second, a threshold sweep for at least one representative blocking defense such as PromptLevelSanitization. Third, a comparison between conclusions drawn from the composite score and conclusions drawn from the primary metric APS alone.

These ablations are feasible within the current codebase because the relevant harnesses are already implemented in `persistbench/ablation/weight_ablation.py`. A companion threshold sweep is implemented in `persistbench/ablation/threshold_sweep.py`. Together, they demonstrate that PersistBench is not only reproducible, but also analytically well-calibrated.

11 Limitations

PersistBench is a controlled research benchmark, not a production security platform or an empirical study of deployed agent systems. The strongest reviewer concerns therefore center on scope, realism, and interpretability. We state those limitations explicitly here.

Synthetic scenarios rather than real-world traces. All 77 benchmark scenarios are synthetic, template-driven, and manually reviewed. This is a deliberate methodological choice that prioritizes reproducibility, precise threat-model specification, and controlled comparison across defenses and backends. The tradeoff is that the benchmark does not establish how closely the observed rankings match real-world deployment behavior. The adversarial content is

grounded in realistic attack patterns, but it is not sourced from real-world attack traces. Whether the benchmark's defense rankings generalize to deployed systems therefore requires future validation with live systems.

EchoBackend as oracle baseline. The default reproducibility path relies on `EchoBackend`, which is useful because it exposes deterministic ground truth for replay, metric computation, and artifact generation. Its limitation is equally important: EchoBackend does not reproduce the full variability of live hosted models and does not by itself provide a behaviorally realistic retrieval or generation environment. In this mode, APS primarily reflects whether defenses block or allow fragment writes under the benchmark trace, effectively measuring interception at the memory-write hook. Qdrant-backed retrieval mode is more realistic for semantic recall, but it is less deterministic and depends on the vector backend configuration. EchoBackend results are best interpreted as a reproducible benchmark baseline, not as a direct proxy for production attack success.

Partial forgetting coverage in the default configuration. Although PersistBench implements FVS-1–15, the default benchmark configuration does not execute all fifteen pathways equally. Under the standard EchoBackend configuration, 10 of 15 tests execute: FVS-1–5 and FVS-11–15. Archive, consolidation, and specialized semantic-probing checks in FVS-6–10 depend on optional backends. When these are absent, the validator records explicit `SKIPPED:*` states. Reported forgetting pass rates must therefore use an effective denominator and disclose which pathways were actually exercised. The current benchmark does not provide complete forgetting coverage under every backend configuration.

Limited live-model evidence. The framework supports Anthropic and OpenAI backends, but the current draft paper is still anchored primarily in the completed deterministic benchmark matrix. This substantially strengthens cross-suite breadth, but the most reproducible results are still the least behaviorally realistic. Live-model experiments would strengthen claims about utility-security tradeoffs, behavioral drift, and retrieval realism, but they introduce temporal drift and non-determinism that reduce exact reproducibility.

Long-horizon evaluation remains limited. The replay engine itself is not restricted to short trajectories, but the current benchmark suite uses a bounded session range of 6–15 sessions per scenario as a design point rather than evaluating truly long-lived memory over dozens of sessions. Longer-horizon studies are possible within the same infrastructure, but they are not yet the empirical focus of the present benchmark release.

Defense set breadth. Seven defense configurations are sufficient to demonstrate benchmark differentiation, but they are not an exhaustive survey of the defense design space. The framework is intentionally extensible through the defense hook protocol, so the current registry is best read as a benchmark calibration set rather than as a complete defense survey.

Heuristic and saturated signals. Some reported observability metrics remain exploratory. In particular, CRA is heuristic and does not carry primary conclusions. Likewise, UPS is saturated at 1.0 in the completed canonical deterministic matrix, which limits its usefulness for discussing practical utility-security tradeoffs without live-model or false-positive intervention data.

12 Ethics

PersistBench studies a dual-use security problem. The benchmark is intended to support defensive evaluation, reproducible measurement, and community extension of attack and defense scenarios. The work does not target real deployed systems, does not involve human subjects, and is released to improve evaluation rigor for memory-enabled agents.

13 Conclusion

PersistBench contributes a reproducible benchmark and evaluation methodology for persistent adversarial attacks in memory-enabled LLM agents. Its value lies in controlled longitudinal evaluation: seeded replay trajectories, provenance-aware analysis, forgetting validation, and artifact-backed reproducibility under a shared defense interface. The scenario suite, canonical deterministic matrix, pre-seeded database, exported result snapshots, and dashboard make it possible for other researchers to inspect, reproduce, and extend the evaluation without rebuilding the infrastructure from scratch.

References

[1] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. arXiv:2407.12784 [cs.LG] doi:10.48550/arXiv.2407.12784

[2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352 [cs.CR] doi:10.48550/arXiv.2406.13352

[3] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory Injection Attacks on LLM Agents via Query-Only Interaction. arXiv:2503.03704 [cs.LG] doi:10.48550/arXiv.2503.03704

[4] Ronen Eldan and Mark Russinovich. 2023. Who's Harry Potter? Approximate Unlearning in LLMs. arXiv:2310.02238 [cs.LG] doi:10.48550/arXiv.2310.02238

[5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173 [cs.CR] doi:10.48550/arXiv.2302.12173

[6] Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens Van Der Maaten. 2020. Certified Data Removal from Machine Learning Models. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 3832–3842. https://proceedings.mlr.press/v119/guo20c.html

[7] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720 [cs.CR] doi:10.48550/arXiv.2403.14720

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2310.06770

[9] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yuhui Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bonnie Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Daniel Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, and Yuta Koreeda. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023). doi:10.48550/arXiv.2211.09110

[10] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating

LLMs as Agents. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2308.03688

[11] Aengus Lynch, Phillip Guo, Aidan Ewart, Stephen Casper, and Dylan Hadfield-Menell. 2024. Eight Methods to Evaluate Robust Unlearning in LLMs. arXiv:2402.16835 [cs.CL] doi:10.48550/arXiv.2402.16835

[12] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. arXiv:2402.04249 [cs.CR] doi:10.48550/arXiv.2402.04249

[13] Seth Neel, Aaron Roth, and Saeed Sharifi-Malvajerdi. 2021. Descent-to-Delete: Gradient-Based Methods for Machine Unlearning. In *Proceedings of the 32nd International Conference on Algorithmic Learning Theory (Proceedings of Machine Learning Research, Vol. 132)*. PMLR, 931–962. https://proceedings.mlr.press/v132/neel21a.html

[14] Robert O'Callahan, Chris Jones, Nathan Froyd, Kyle Huey, Albert Noll, and Nimrod Partush. 2017. Engineering Record And Replay For Deployability. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. 377–389. https://www.usenix.org/conference/atc17/technical-sessions/presentation/ocallahan

[15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] doi:10.48550/arXiv.2310.08560

[16] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. doi:10.48550/arXiv.2304.03442

[17] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. 2017. Practical Whole-System Provenance Capture. In *Proceedings of the 2017 ACM Symposium on Cloud Computing*. 405–418. doi:10.48550/arXiv.1711.05296

[18] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789 [cs.AI] doi:10.48550/arXiv.2307.16789

[19] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] doi:10.48550/arXiv.2308.08155

[20] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2025. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. doi:10.1145/3690624.3709179

[21] Kaijie Zhu, Qinlin Zhao, Hao Chen, Jindong Wang, and Xing Xie. 2024. PromptBench: A Unified Library for Evaluation of Large Language Models. *Journal of Machine Learning Research* 25, 254 (2024), 1–22. doi:10.48550/arXiv.2312.07910